

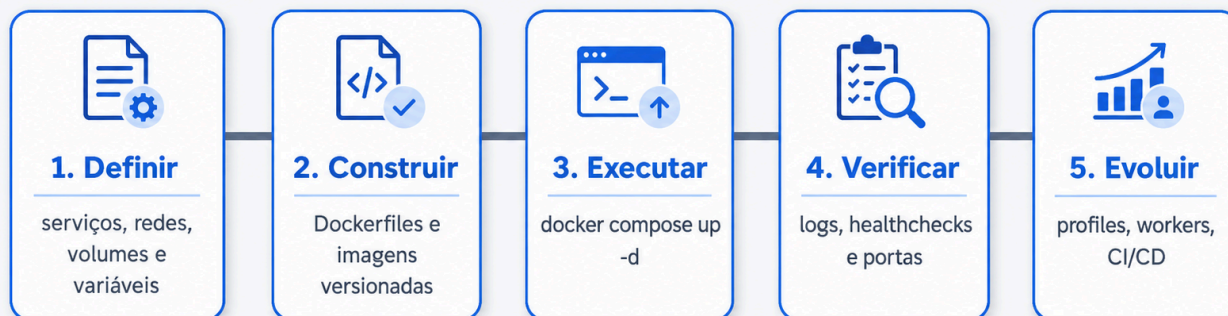


Docker e Docker Compose para ambientes profissionais de desenvolvimento

Windows 11, Linux e macOS | Front, Back, Banco, Redis e Mensageria

Material acadêmico com foco em prática profissional, elaborado por Carlos Uchôa e William Sestito — Agosto de 2026

Ciclo profissional de trabalho com Docker Compose



Ideia-chave: Compose não é apenas “subir containers”; é uma forma de tornar o ambiente executável, revisável e padronizado para a equipe.

Sumário do material

Seção	Foco
1. Visão geral	Por que containers importam em ADS e no mercado
2. Fundamentos	Imagem, container, volume, rede, porta, Dockerfile e Compose
3. Instalação e ambiente	Windows 11 sem WSL, Linux e macOS
4. Boas práticas	Segurança, versionamento, variáveis, logs e healthchecks
5. Cenário 1	VueJS + NodeJS + MySQL + Redis + RabbitMQ
6. Cenário 2	ReactJS + PostgreSQL + Node/Express/Inertia + Redis + RabbitMQ
7. Cenário 3	Atividade de consolidação com requisitos mínimos
8. Atividades parciais	Teóricas e práticas por etapa
9. Rubrica	Critérios de avaliação
10. Referências	Bibliografia e documentação oficial

1. Visão geral e objetivos de aprendizagem

Este material foi elaborado para acadêmicos de Análise e Desenvolvimento de Sistemas com foco em uso profissional de Docker e Docker Compose. A proposta não é ensinar a criar do zero aplicações Vue, React, Node ou bancos de dados, mas sim compreender como estruturar ambientes executáveis, reproduzíveis e próximos da rotina real de times de desenvolvimento.

Ao final, o acadêmico deverá ser capaz de ler, explicar, executar, ajustar e documentar uma arquitetura local composta por frontend, backend, banco de dados, cache Redis e mensageria. Essa competência é importante porque reduz o problema clássico de “funciona na minha máquina”, melhora a integração entre equipes, viabiliza onboarding mais rápido e aproxima o ambiente acadêmico do ambiente usado em empresas.

Competência central: transformar uma arquitetura desenhada no quadro em um ambiente versionado que qualquer integrante da equipe consiga executar com poucos comandos.

Objetivo	Evidência esperada
Compreender containers	Explicar a diferença entre imagem, container, volume, rede e porta.
Executar Compose	Subir e derrubar uma pilha multi-container de forma controlada.
Diagnosticar problemas	Usar logs, ps, exec, inspect e healthchecks.
Documentar profissionalmente	Entregar README, .env.example, diagrama e evidências.
Pensar em produção	Separar desenvolvimento, build, runtime, dados persistentes e segredos.

1.1 Aplicabilidade profissional

Em uma software house, Docker Compose é usado para padronizar o ambiente local e reduzir conflitos de versão entre Node, banco de dados, cache, filas e dependências do sistema operacional.

Em squads de produto, o Compose permite que frontend, backend e banco sejam integrados antes da entrega em homologação, identificando erros de configuração, portas, variáveis e dependências.

Em projetos acadêmicos, o Compose ajuda a equipe a documentar decisões de arquitetura e permite que professor, banca e colegas reproduzam o ambiente sem instalar manualmente todos os serviços.

2. Fundamentos conceituais

Docker trabalha com a ideia de empacotar uma aplicação e suas dependências em imagens executáveis. Um container é uma instância isolada dessa imagem. O isolamento não substitui conhecimento de sistema operacional, rede ou segurança; ele organiza a execução para que a equipe controle versões e dependências com mais previsibilidade.

Termo	Definição didática	Uso real
Imagem	Modelo imutável com sistema base, runtime e arquivos necessários.	Versionar builds, publicar no registry, promover entre ambientes.
Container	Processo isolado executado a partir de uma imagem.	Rodar API, banco, worker, broker e serviços auxiliares.
Dockerfile	Receita declarativa para construir imagem.	Padronizar build, instalar dependências e preparar runtime.
Volume	Área persistente gerenciada pelo Docker.	Guardar dados do banco, cache persistente e arquivos de desenvolvimento.
Network	Rede virtual entre containers.	Permitir que api acesse db pelo nome do serviço, sem IP fixo.
Compose	Arquivo YAML para orquestrar múltiplos containers.	Subir pilha completa local com um comando.
Healthcheck	Teste de saúde do container.	Evitar iniciar dependentes antes de serviços críticos estarem prontos.

2.1 Docker Compose como contrato técnico

Um arquivo docker-compose.yml deve ser entendido como contrato técnico entre membros da equipe. Ele responde: quais serviços existem, quais portas são expostas, quais variáveis são necessárias, quais dados persistem, quais dependências existem e como diagnosticar o ambiente.

```
services:
  api:
    build: ./backend
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: development
    depends_on:
      db:
        condition: service_healthy

networks:
  app_net:

volumes:
  db_data:
```

No exemplo, api é o serviço de aplicação, db seria uma dependência com healthcheck, app_net representa a rede lógica e db_data preserva dados. Em equipes reais, esse arquivo deve ser revisado em pull requests com o mesmo cuidado dado ao código-fonte.

3. Preparação do ambiente por sistema operacional

A proposta do laboratório é que todos usem Docker Desktop ou Docker Engine com Docker Compose v2. Como requisito acadêmico, a trilha Windows usa Docker Desktop no próprio

Windows 11, sem WSL. Isso exige atenção à edição do Windows e ao backend de virtualização disponível.

Importante para Windows: sem WSL, usar Docker Desktop com backend Hyper-V em Windows 11 Pro, Education ou Enterprise. O Windows Home normalmente depende de WSL 2 para Linux containers e não atende a esta trilha sem ajustes de laboratório.

Sistema	Caminho recomendado	Observações de laboratório
Windows 11 sem WSL	Docker Desktop com Hyper-V e recurso Containers habilitados.	Preferir Pro/Education/Enterprise; executar PowerShell como administrador para habilitar recursos.
Linux	Docker Engine + Docker Compose plugin.	Adicionar usuário ao grupo docker apenas em ambiente controlado; reiniciar sessão.
macOS	Docker Desktop para Apple Silicon ou Intel.	Verificar arquitetura da imagem: amd64 vs arm64; usar tags multi-arch quando possível.

3.1 Windows 11 sem WSL

No Windows 11, a abordagem sem WSL depende do Hyper-V. Em laboratório, validar previamente a edição do Windows, virtualização habilitada na BIOS/UEFI, recursos Hyper-V e Containers ativados e Docker Desktop configurado para não usar engine baseada em WSL.

```
# PowerShell como Administrador
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All
Enable-WindowsOptionalFeature -Online -FeatureName Containers -All

# Após reiniciar, validar:
docker version
docker compose version
docker run --rm hello-world
```

Para os cenários deste material, usar Linux containers no Docker Desktop. As imagens oficiais de Node, MySQL, PostgreSQL, Redis e RabbitMQ são distribuídas majoritariamente para ambientes Linux; o Docker Desktop abstrai a VM necessária por trás do mecanismo escolhido.

3.2 Linux

```
# Exemplo Debian/Ubuntu - adaptar conforme a distribuição
docker version
docker compose version

# Teste rápido
docker run --rm hello-world

# Diagnóstico comum
```

```
docker info
docker ps -a
```

Em Linux, o Docker Engine roda de forma mais direta sobre o kernel do sistema. Isso facilita compatibilidade com volumes e desempenho de I/O, mas exige cuidado com permissões de arquivos criados por containers, principalmente em projetos Node com `node_modules` e `bind mounts`.

3.3 macOS

```
docker version
docker compose version
docker run --rm --platform linux/arm64 hello-world
# Em Macs Intel, normalmente linux/amd64. Em Apple Silicon, preferir imagens multi-arch.
```

No macOS, Docker Desktop também usa uma VM Linux. Em Apple Silicon, algumas imagens antigas podem não ter suporte `arm64`. A equipe deve registrar no README quando for necessário usar `platform: linux/amd64`, sabendo que isso pode reduzir desempenho por emulação.

4. Comandos essenciais e diagnóstico

Comando	Quando usar
<code>docker compose up -d</code>	Subir a pilha em segundo plano.
<code>docker compose down</code>	Derrubar containers e rede do projeto.
<code>docker compose down -v</code>	Derrubar e remover volumes; apaga dados locais.
<code>docker compose ps</code>	Ver status e portas.
<code>docker compose logs -f api</code>	Acompanhar logs de um serviço.
<code>docker compose exec api sh</code>	Entrar no container em execução.
<code>docker compose build --no-cache api</code>	Reconstruir imagem sem cache.
<code>docker inspect <container></code>	Investigar rede, mounts, variáveis e healthcheck.
<code>docker volume ls</code>	Listar volumes persistentes.
<code>docker network ls</code>	Listar redes Docker.

Regra profissional: antes de "apagar tudo", coletar evidências: `docker compose ps`, logs do serviço, `docker inspect` e conteúdo do `.env` usado.

5. Boas práticas profissionais

Prática	Justificativa
Usar <code>.env.example</code> versionado	Mostra quais variáveis são necessárias sem expor segredos.
Não versionar <code>.env</code> real	Evita vazamento de senhas, tokens e chaves.
Fixar versões principais	Reduz surpresa por mudança de <code>latest</code> .
Separar build e runtime	Gera imagens menores e mais seguras.
Criar healthchecks	Melhora diagnóstico e ordem de inicialização.
Usar volumes nomeados para bancos	Preserva dados entre reinícios.
Usar bind mounts no código em desenvolvimento	Permite hot reload sem rebuild constante.

Evitar rodar tudo como root quando possível	Reduz risco e aproxima de produção.
Documentar portas e credenciais locais	Facilita onboarding e avaliação acadêmica.
Remover recursos não usados	Controla consumo de disco e memória.

5.1 Dockerfile de referência para Node em desenvolvimento

```
FROM node:24-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000
CMD ["npm", "run", "dev"]
```

Para desenvolvimento, a prioridade é feedback rápido. Em produção, a imagem deve ser mais enxuta, instalar apenas dependências necessárias e preferencialmente usar multi-stage build. A prática de multi-stage reduz tamanho de imagem e superfície de ataque ao separar etapas de build e execução.

6. Cenário 1 — VueJS + NodeJS + MySQL + Redis + RabbitMQ

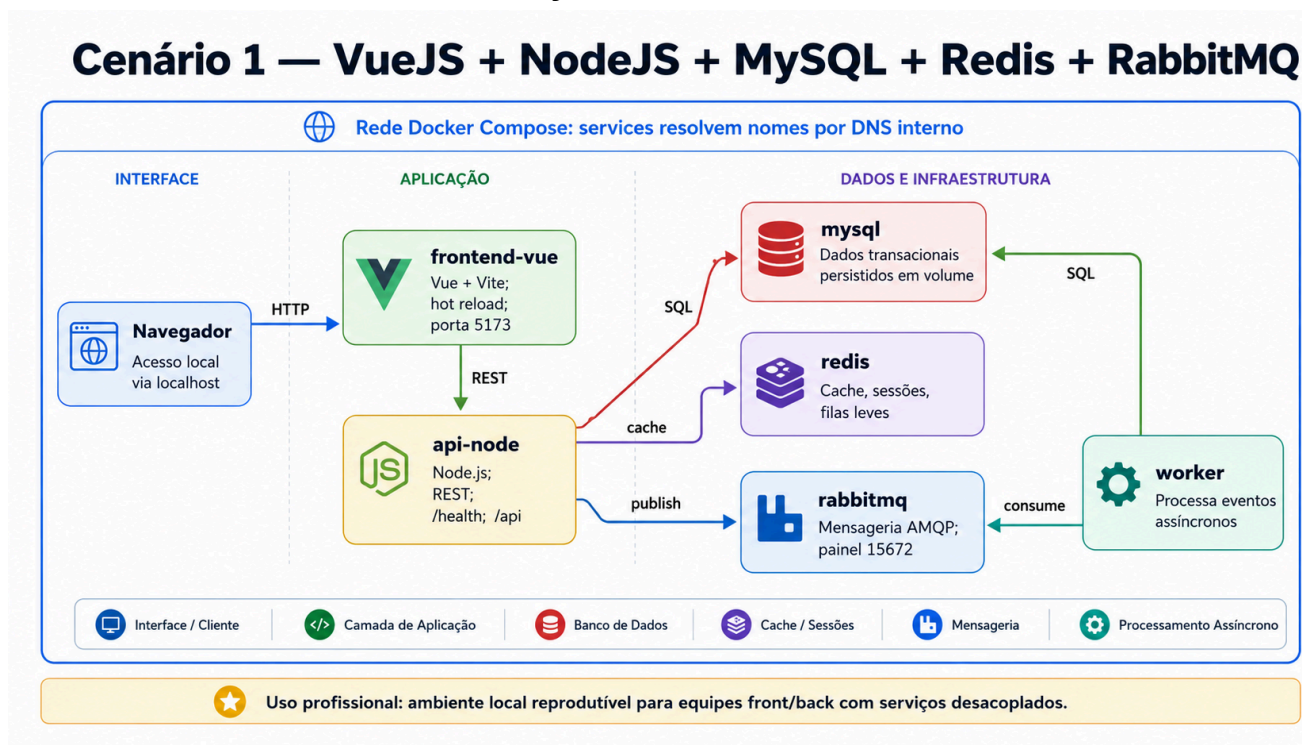


Figura 1. Estrutura de containers do cenário 1.

Este cenário representa uma aplicação web clássica: frontend VueJS em desenvolvimento, backend NodeJS, banco MySQL, Redis para cache/sessões e RabbitMQ para mensageria. O foco é a estrutura de containers, não a implementação das funcionalidades.

Serviço	Responsabilidade	Porta local sugerida
frontend	Executar Vue/Vite em modo desenvolvimento.	5173
api	Expor endpoints REST e integrar dependências.	3000
mysql	Persistir dados transacionais.	3306
redis	Cache, sessões e filas simples.	6379
rabbitmq	Broker AMQP e painel de gerenciamento.	5672 / 15672
worker	Consumir mensagens e executar tarefas assíncronas.	sem porta pública

6.1 Estrutura de pastas sugerida

```

cenario-1-vue-node-mysql/
├── docker-compose.yml
├── .env.example
├── frontend/
│   └── Dockerfile
├── backend/
│   └── Dockerfile
├── worker/
│   └── Dockerfile
└── database/
    └── init.sql

```

6.2 Compose de referência

```

name: ads-cenario-1

services:
  frontend:
    build: ./frontend
    command: npm run dev -- --host 0.0.0.0
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - frontend_node_modules:/app/node_modules
    depends_on:
      - api
    networks: [app_net]

  api:
    build: ./backend
    command: npm run dev
    ports:
      - "3000:3000"
    env_file: .env
    volumes:
      - ./backend:/app
      - backend_node_modules:/app/node_modules
    depends_on:
      mysql:
        condition: service_healthy

```



```
redis:
  condition: service_started
rabbitmq:
  condition: service_healthy
networks: [app_net]

mysql:
  image: mysql:8.4
  environment:
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  ports:
    - "3306:3306"
  volumes:
    - mysql_data:/var/lib/mysql
    - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s
    timeout: 5s
    retries: 10
  networks: [app_net]

redis:
  image: redis:7.4-alpine
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  networks: [app_net]

rabbitmq:
  image: rabbitmq:4-management
  ports:
    - "5672:5672"
    - "15672:15672"
  environment:
    RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER}
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD}
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 10s
    timeout: 5s
    retries: 10
  networks: [app_net]

worker:
  build: ./worker
  command: npm run worker
  env_file: .env
```

```

depends_on:
  rabbitmq:
    condition: service_healthy
  mysql:
    condition: service_healthy
networks: [app_net]

networks:
  app_net:

volumes:
  mysql_data:
  redis_data:
  frontend_node_modules:
  Backend_node_modules:

```

6.3 Atividades parciais do cenário 1

Tipo	Atividade	Produto esperado
Teórica	Explique por que api usa mysql, redis e rabbitmq por nome de serviço, e não por localhost.	Resposta curta com diagrama de rede.
Prática	Suba a pilha, liste containers, abra logs da api e do rabbitmq.	Prints ou transcrição dos comandos.
Prática	Remova apenas containers com docker compose down e suba novamente. Verifique se dados do MySQL persistem.	Evidência do volume.
Análise	Compare Redis e RabbitMQ: quando usar cache e quando usar fila?	Tabela comparativa com exemplo real.

7. Cenário 2 — ReactJS + PostgreSQL + Node/Express/Inertia + Redis + RabbitMQ

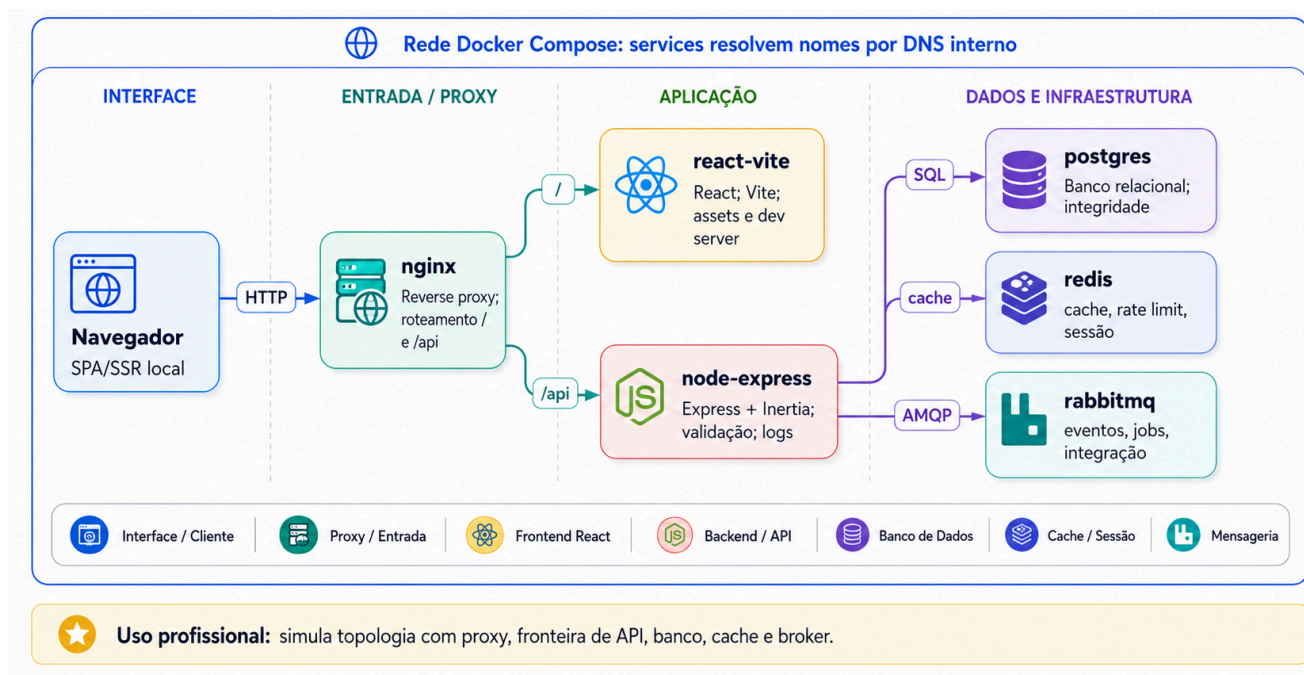


Figura 2. Estrutura de containers do cenário 2.

O segundo cenário adiciona um reverse proxy Nginx para aproximar o laboratório de uma topologia comum em ambientes profissionais. O frontend React e a API Node/Express podem ser roteados pelo proxy, enquanto PostgreSQL, Redis e RabbitMQ permanecem em rede interna.

Serviço	Responsabilidade	Observação
nginx	Entrada única da aplicação.	Roteia / para frontend e /api para backend.
react	Frontend React/Vite.	Hot reload em desenvolvimento.
api	Node.js + Express + Inertia e middlewares.	Validação, logs, autenticação local e integração.
postgres	Banco relacional.	Volume persistente e healthcheck.
redis	Cache, sessão e rate limit.	Pode ser usado por API e worker.
rabbitmq	Broker de mensagens.	Eventos de domínio e tarefas assíncronas.
pgadmin	Ferramenta opcional para inspeção.	Usar profile tools para não subir sempre.

7.1 Estrutura de pastas sugerida

```
cenario-2-react-express-postgres/  
├─ docker-compose.yml  
├─ .env.example  
├─ nginx/  
│   └─ default.conf  
├─ frontend/  
│   └─ Dockerfile  
├─ backend/  
│   └─ Dockerfile  
└─ database/  
    └─ init.sql
```

7.2 Compose de referência

```
name: ads-cenario-2
```

```
services:
```

```
  proxy:
```

```
    image: nginx:1.27-alpine
```

```
    ports:
```

```
      - "8080:80"
```

```
    volumes:
```

```
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
```

```
    depends_on:
```

```
      - frontend
```

```
      - api
```

```
    networks: [app_net]
```

```
  frontend:
```

```
    build: ./frontend
```

```
    command: npm run dev -- --host 0.0.0.0
```

```
    volumes:
```

```
      - ./frontend:/app
```

```
      - react_node_modules:/app/node_modules
```

```
    expose:
```

```
      - "5173"
```

```
    networks: [app_net]
```

```
  api:
```

```
    build: ./backend
```

```
    command: npm run dev
```

```
    env_file: .env
```

```
    volumes:
```

```
      - ./backend:/app
```

```
      - api_node_modules:/app/node_modules
```

```
    expose:
```

```
      - "3000"
```

```
    depends_on:
```

```
      postgres:
```

```
        condition: service_healthy
```

```
      redis:
```

```
    condition: service_started
  rabbitmq:
    condition: service_healthy
  networks: [app_net]

postgres:
  image: postgres:17-alpine
  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
    interval: 10s
    timeout: 5s
    retries: 10
  networks: [app_net]

redis:
  image: redis:7.4-alpine
  networks: [app_net]

rabbitmq:
  image: rabbitmq:4-management
  ports:
    - "15672:15672"
  environment:
    RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER}
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD}
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "ping"]
    interval: 10s
    timeout: 5s
    retries: 10
  networks: [app_net]

pgadmin:
  image: dpage/pgadmin4:8
  profiles: ["tools"]
  ports:
    - "5050:80"
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@local.test
    PGADMIN_DEFAULT_PASSWORD: admin
  networks: [app_net]

networks:
  app_net:
```

```
volumes:
  postgres_data:
  react_node_modules:
  Api_node_modules:
```

7.3 Exemplo de configuração do Nginx

```
server {
    listen 80;

    location / {
        proxy_pass http://frontend:5173;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        proxy_pass http://api:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

7.4 Atividades parciais do cenário 2

Tipo	Atividade	Produto esperado
Teórica	Explique a diferença entre ports e expose no Compose.	Resposta com exemplo do proxy.
Prática	Suba o cenário 2 e acesse a aplicação via <code>http://localhost:8080</code> .	Evidência do acesso.
Prática	Suba ferramentas com <code>docker compose --profile tools up -d</code> .	PgAdmin acessível na porta 5050.
Análise	Justifique por que PostgreSQL não precisa publicar porta para o host quando apenas a API acessa o banco.	Texto técnico curto.

8. Cenário 3 — Atividade de consolidação - Projeto aplicado com Docker Compose

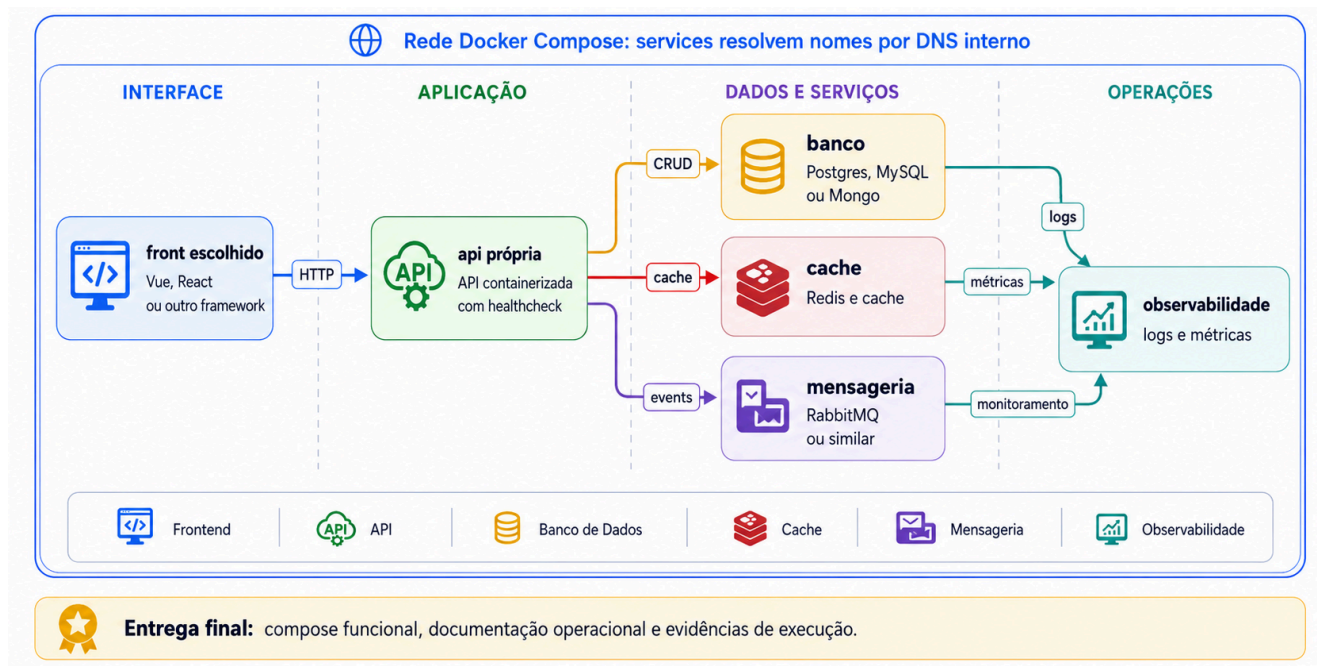


Figura 3. Desafio final com arquitetura escolhida pela equipe.

No terceiro cenário, a equipe deve aplicar os conceitos sem receber um docker-compose.yml completo. O objetivo é forçar tomada de decisão, pesquisa técnica, documentação e diagnóstico. A equipe pode escolher Vue ou React no frontend, Node/Express ou outro framework compatível no backend, e deve obrigatoriamente incluir banco, Redis e mensageria.

Requisito mínimo	Critério
Frontend containerizado	Deve executar por Compose e ser acessível pelo navegador.
Backend containerizado	Deve expor endpoint /health ou equivalente.
Banco persistente	Deve usar volume nomeado e variáveis no .env.
Redis	Deve estar na rede interna e ser referenciado pelo nome do serviço.
Mensageria	RabbitMQ ou serviço equivalente com healthcheck.
Worker	Container separado para consumir fila ou simular consumo.
README	Instruções de execução para Windows 11 sem WSL, Linux e macOS.
Evidências	Prints ou logs de execução, healthcheck e comunicação entre serviços.

8.1 Entrega final de fixação

Item	Descrição	Peso sugerido
docker-compose.yml	Serviços, redes, volumes, env_file, depends_on e healthchecks coerentes.	30%
Dockerfiles	Imagens funcionais para front, back e worker.	15%
.env.example	Variáveis sem segredos reais e documentação mínima.	10%
README.md	Passo a passo, portas, troubleshooting e comandos úteis.	20%
Demonstração	Execução em sala ou vídeo curto demonstrando a pilha.	15%
Análise crítica	O que seria diferente em produção?	10%

8.2 Questões norteadoras para a equipe

- Quais serviços precisam ser acessados pelo host e quais devem permanecer apenas na rede interna?
- Quais dados precisam sobreviver ao docker compose down?
- Quais variáveis podem ser públicas no .env.example e quais seriam segredos?
- Como a equipe provará que backend conversa com banco, Redis e mensageria?
- Como o ambiente será executado por colegas usando Windows 11 sem WSL, Linux e macOS?

9. Sequência didática sugerida

Aula/Etapa	Conteúdo	Atividade parcial
1	Conceitos de imagem, container, volume, rede e porta.	Mapa conceitual individual.
2	Instalação e validação em Windows 11 sem WSL, Linux e macOS.	Checklist de ambiente.
3	Primeiro Compose com dois serviços.	Subir api + banco e registrar logs.
4	Volumes, redes e variáveis.	Persistência controlada e .env.example.
5	Cenário 1.	Executar e explicar arquitetura.
6	Redis e mensageria.	Comparar cache, fila e evento.
7	Cenário 2 com proxy e profiles.	Roteamento pelo Nginx e profile tools.
8	Healthchecks e troubleshooting.	Resolver erro proposto pelo professor.
9	Cenário 3.	Construção em equipe.
10	Apresentação final.	Demonstração e rubrica.

10. Banco de atividades teóricas e práticas

10.1 Atividades teóricas

1. Diferencie imagem e container usando uma analogia técnica e um exemplo com Node.js.
2. Explique porque os containers não substituem boas práticas de segurança, versionamento e observabilidade.
3. Compare volume nomeado e bind mount. Em qual caso cada um é mais adequado?
4. Explique porque localhost dentro de um container não representa o host nem outro container.
5. Explique a diferença entre Redis como cache e RabbitMQ como broker de mensagens.
6. Discuta por que o latest pode ser perigoso em ambientes profissionais.
7. Explique como Compose melhora o onboarding de novos desenvolvedores.
8. Liste três riscos ao versionar .env com credenciais reais.

10.2 Atividades práticas

Nº	Atividade	Critério de aceite
P1	Criar .env.example para o cenário 1.	Sem senhas reais; nomes coerentes com compose.
P2	Executar docker compose config.	Arquivo validado sem erro de sintaxe.
P3	Subir cenário 1 e coletar logs.	Logs de api, mysql e rabbitmq apresentados.
P4	Executar comando dentro da api.	docker compose exec api sh ou equivalente.
P5	Simular perda de container sem perder volume.	Dados persistem após recriação.
P6	Ativar profile tools no cenário 2.	PgAdmin executando apenas quando solicitado.
P7	Adicionar healthcheck em serviço escolhido.	docker compose ps mostra status healthy.
P8	Documentar troubleshooting.	README contém pelo menos três erros comuns e soluções.

11. Troubleshooting orientado

Sintoma	Causa provável	Ação
Porta já está em uso	Outro serviço no host usa a porta.	Trocar mapeamento host:container ou encerrar processo conflitante.
Banco reinicia em loop	Variáveis obrigatórias ausentes ou volume com estado incompatível.	Validar .env, logs e, em laboratório, recriar volume se permitido.
API não conecta no banco	Uso incorreto de localhost.	Usar nome do serviço: mysql ou postgres.
Permissão em node_modules	Conflito entre host e container.	Usar volume nomeado para /app/node_modules.
RabbitMQ não abre painel	Porta 15672 não publicada ou imagem sem management.	Usar rabbitmq:4-management e mapear 15672.
PostgreSQL não aceita conexão	Serviço ainda inicializando.	Usar healthcheck com pg_isready e depends on condition.
Windows sem WSL não sobe	Hyper-V/Containers indisponíveis ou Windows Home.	Validar edição do Windows e recursos opcionais.
Imagem incompatível em Mac Apple Silicon	Imagem sem arm64.	Usar imagem multi-arch ou platform com consciência de desempenho.

12. Rubrica de avaliação

Critério	Excelente	Adequado	Insuficiente
Arquitetura	Serviços bem separados, rede interna clara, portas mínimas.	Serviços sobem, mas com pouca justificativa.	Tudo exposto, acoplado ou sem coerência.
Compose	Usa volumes, env_file, healthchecks, profiles e nomes claros.	Usa Compose básico funcional.	Compose com erros, duplicações ou sem persistência.
Diagnóstico	Apresenta logs, ps, exec, inspect e análise de falhas.	Mostra execução básica.	Não consegue explicar falhas.
Documentação	README reproduzível em Windows sem WSL, Linux e macOS.	README parcial.	Instruções ausentes ou dependentes do professor.
Segurança	Não expõe segredos, evita latest, separa variáveis.	Poucos cuidados de segurança.	Credenciais reais ou práticas arriscadas.
Apresentação	Explica decisões técnicas e limitações.	Demonstra funcionamento.	Não conecta prática e conceito.

13. Referências bibliográficas e documentação técnica

As referências abaixo sustentam a abordagem conceitual e prática do material. A documentação oficial deve ser consultada durante as atividades, pois versões de imagens, requisitos de instalação e recursos do Compose evoluem com frequência.

- DOCKER. Docker Desktop on Windows: installation and system requirements. Documentação oficial. Acesso em agosto de 2026.
- DOCKER. Compose file reference: services, networks, volumes and Compose Specification. Documentação oficial. Acesso em agosto de 2026.
- DOCKER. Building best practices and multi-stage builds. Documentação oficial. Acesso em agosto de 2026.
- DOCKER HUB. Node Official Image. Documentação da imagem oficial. Acesso em agosto de 2026.
- DOCKER HUB. MySQL Official Image. Documentação da imagem oficial. Acesso em agosto de 2026.
- DOCKER HUB. PostgreSQL Official Image. Documentação da imagem oficial. Acesso em agosto de 2026.
- REDIS. Run Redis Open Source on Docker. Documentação oficial. Acesso em agosto de 2026.
- RABBITMQ. RabbitMQ Docker image and installation documentation. Acesso em agosto de 2026.
- KANE, Sean P.; MATTHIAS, Karl. Docker: Up & Running. 3. ed. Sebastopol: O'Reilly Media, 2023.
- NICKOLOFF, Jeff; KUENZLI, Stephen. Docker in Action. 2. ed. Shelter Island: Manning, 2019.
- KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John; FORSGREN, Nicole. The DevOps Handbook. 2. ed. Portland: IT Revolution, 2021.
- NEWMAN, Sam. Building Microservices. 2. ed. Sebastopol: O'Reilly Media, 2021.
- KLEPPMANN, Martin. Designing Data-Intensive Applications. Sebastopol: O'Reilly Media, 2017.
- HUMBLE, Jez; FARLEY, David. Continuous Delivery. Boston: Addison-Wesley, 2010.
- NYGARD, Michael T. Release It! 2. ed. Raleigh: Pragmatic Bookshelf, 2018.

Apêndice A — Checklist de entrega do aluno

- docker-compose.yml executa sem erro com docker compose up -d.
- Todos os serviços possuem nomes claros.
- Banco possui volume nomeado.
- Variáveis sensíveis não estão no repositório.
- README explica execução em Windows 11 sem WSL, Linux e macOS.
- Há diagrama ou descrição da arquitetura.
- Há evidências de logs, containers healthy e acesso pelas portas documentadas.
- Há uma seção explicando limitações e diferenças entre desenvolvimento e produção.

Apêndice B — Modelo mínimo de README

```
# Projeto Docker Compose - ADS Unisenai

## Requisitos
- Docker Desktop ou Docker Engine
- Docker Compose v2
- Windows 11 sem WSL: Hyper-V e Containers habilitados

## Como executar
cp .env.example .env
docker compose up -d --build
docker compose ps

## Portas
- Frontend: http://localhost:5173 ou proxy em http://localhost:8080
- API: http://localhost:3000/health
- RabbitMQ: http://localhost:15672

## Diagnóstico
docker compose logs -f api
docker compose exec api sh
docker compose down

## Observações
Não versionar .env real. Para resetar banco em laboratório, usar docker compose
down -v.
```